

DAY 7

Python File Handling, CSV and JSON

Muhammad Ahmad | 60-Day Intensive Data Science Learning Journey | Goal: Data Scientist

Objective: Learn how to create, read, write, append and process files, work with CSV datasets, and store and analyze structured data with JSON.

1. File Handling Basics

File handling lets Python create files, read stored data, write new data, append information, and process data. Common Data Science files include CSV and JSON.

```
file = open("data.txt", "r")
```

Common modes: r = read, w = write/replace, a = append, x = create new file.

2. Reading Files

```
file = open("data.txt", "r")
data = file.read()
print(data)
file.close()
```

read() reads the complete file. close() closes the file.

```
with open("data.txt", "r") as file:
    data = file.read()
print(data)
```

with open() is preferred because Python automatically closes the file.

```
with open("data.txt", "r") as file:
    print(file.readline())
```

```
with open("data.txt", "r") as file:
    lines = file.readlines()
    print(lines)
```

readline() reads one line. readlines() returns the lines as a list.

```
with open("data.txt", "r") as file:
    for line in file:
        print(line.strip())
```

Looping line by line is useful when processing many records. strip() removes surrounding whitespace and newline characters.

3. Writing and Appending

```
with open("students.txt", "w") as file:
    file.write("Ahmad\n")
    file.write("Ali\n")
    file.write("Usman\n")
```

w creates or replaces content. Be careful because existing content can be overwritten.

```
with open("students.txt", "a") as file:
    file.write("Hamza\n")
```

a adds content at the end.

```
with open("newfile.txt", "x") as file:
    file.write("Hello Ahmad")
```

x creates a new file and fails if it already exists.

4. Practical Text File

```
students = ["Ahmad", "Ali", "Usman", "Hamza"]
```

```
with open("students.txt", "w") as file:
    for student in students:
        file.write(student + "\n")
```

```
with open("students.txt", "r") as file:
    for student in file:
        print(student.strip())
```

5. CSV Files

CSV means Comma-Separated Values. It is widely used for tabular datasets.

```
name,marks,semester
Ahmad,85,7
Ali,72,6
Usman,65,5
Hamza,91,7

import csv

with open("students.csv", "r") as file:
    reader = csv.reader(file)
    for row in reader:
        print(row)
```

csv.reader() returns rows as lists. CSV values are normally read as strings.

```
import csv

with open("students.csv", "r") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row["name"], row["marks"])
```

DictReader uses the header row as dictionary keys, making structured data easier to process.

```
import csv

students = [
    ["Ahmad", 85, 7],
    ["Ali", 72, 6],
    ["Usman", 65, 5]
]

with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["name", "marks", "semester"])
    writer.writerows(students)
```

writerow() writes one row. writerows() writes multiple rows.

6. JSON

JSON means JavaScript Object Notation. It is widely used by APIs, web applications, configuration files, and data exchange systems.

```
{
  "name": "Ahmad",
  "age": 22,
  "cgpa": 3.23
}
```

JSON looks similar to a Python dictionary, but JSON is a data format.

```
import json

student = {"name": "Ahmad", "age": 22, "cgpa": 3.23}
json_data = json.dumps(student, indent=4)
print(json_data)
```

dumps() converts Python data to a JSON string. indent=4 makes it readable.

```
import json

json_data = '{"name": "Ahmad", "cgpa": 3.23}'
student = json.loads(json_data)
print(student["name"])
print(student["cgpa"])
```

loads() converts a JSON string into a Python object.

```
import json

student = {"name": "Ahmad", "age": 22, "cgpa": 3.23}

with open("student.json", "w") as file:
    json.dump(student, file, indent=4)
```

dump() writes Python data directly to a JSON file.

```
import json
```

```

with open("student.json", "r") as file:
    student = json.load(file)

print(student["name"])
print(student["cgpa"])

```

load() reads JSON from a file. Remember: dumps/loads work with strings, dump/load work with files.

7. JSON List of Students

```

import json

students = [
    {"name": "Ahmad", "marks": 85, "cgpa": 3.23, "semester": 7},
    {"name": "Ali", "marks": 72, "cgpa": 3.10, "semester": 6},
    {"name": "Hamza", "marks": 91, "cgpa": 3.60, "semester": 7}
]

with open("students.json", "w") as file:
    json.dump(students, file, indent=4)

with open("students.json", "r") as file:
    students = json.load(file)

for student in students:
    print(student["name"], student["marks"], student["cgpa"])

```

8. Analyze JSON Data

```

high_scorers = [s for s in students if s["marks"] >= 80]
print(high_scorers)

highest_marks = max(students, key=lambda s: s["marks"])
print(highest_marks)

marks = [s["marks"] for s in students]
print("Average:", sum(marks) / len(marks))

```

This combines Day 6 lists and dictionaries with Day 7 file handling.

9. Error Awareness

```

try:
    with open("missing.txt", "r") as file:
        data = file.read()
except FileNotFoundError:
    print("File was not found.")

```

Exception handling will be studied in more depth later.

10. Data Science Connection

Real datasets often arrive as CSV or JSON. The workflow is: File → Read → Inspect → Clean → Analyze → Visualize → Machine Learning. Later, Pandas will make CSV and JSON processing much more powerful.

11. Practice Tasks and Solutions

Try every task yourself first, then compare your answer with these complete solutions.

Task 1: Create students.txt

```

students = ["Ahmad", "Ali", "Usman", "Hamza", "Bilal"]
with open("students.txt", "w") as file:
    for student in students:
        file.write(student + "\n")

```

Task 2: Read students.txt

```

with open("students.txt", "r") as file:
    for student in file:
        print(student.strip())

```

Task 3: Append one student

```

with open("students.txt", "a") as file:
    file.write("Hassan\n")

```

Task 4: Create CSV

```
import csv
students = [
    ["Ahmad", 85, 7], ["Ali", 72, 6], ["Usman", 65, 5],
    ["Hamza", 91, 7], ["Bilal", 78, 6]
]
with open("students.csv", "w", newline="") as file:
    writer = csv.writer(file)
    writer.writerow(["name", "marks", "semester"])
    writer.writerows(students)
```

Task 5: Read CSV with DictReader

```
import csv
with open("students.csv", "r") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row["name"], row["marks"], row["semester"])
```

Task 6: Create JSON

```
import json
students = [
    {"name": "Ahmad", "marks": 85, "cgpa": 3.23, "semester": 7},
    {"name": "Ali", "marks": 72, "cgpa": 3.10, "semester": 6},
    {"name": "Usman", "marks": 65, "cgpa": 2.90, "semester": 5},
    {"name": "Hamza", "marks": 91, "cgpa": 3.60, "semester": 7},
    {"name": "Bilal", "marks": 78, "cgpa": 3.05, "semester": 6}
]
with open("students.json", "w") as file:
    json.dump(students, file, indent=4)
```

Task 7: Average marks from JSON

```
import json
with open("students.json", "r") as file:
    students = json.load(file)
marks = [s["marks"] for s in students]
print("Average marks:", sum(marks) / len(marks))
```

Task 8: Highest CGPA

```
highest_cgpa = max(students, key=lambda s: s["cgpa"])
print(highest_cgpa)
```

Task 9: CGPA greater than 3.0

```
students_above_3 = [s for s in students if s["cgpa"] > 3.0]
print(students_above_3)
```

Task 10: Complete Student Data Manager

```
import json

students = [
    {"name": "Ahmad", "marks": 85, "cgpa": 3.23, "semester": 7},
    {"name": "Ali", "marks": 72, "cgpa": 3.10, "semester": 6},
    {"name": "Usman", "marks": 65, "cgpa": 2.90, "semester": 5},
    {"name": "Hamza", "marks": 91, "cgpa": 3.60, "semester": 7},
    {"name": "Bilal", "marks": 78, "cgpa": 3.05, "semester": 6}
]

with open("students.json", "w") as file:
    json.dump(students, file, indent=4)

with open("students.json", "r") as file:
    students = json.load(file)

for student in students:
    print(student)

marks = [s["marks"] for s in students]
print("Average Marks:", sum(marks) / len(marks))
print("Highest Marks:", max(marks))
print("Highest CGPA:", max(students, key=lambda s: s["cgpa"]))
print("CGPA >= 3.0:", [s for s in students if s["cgpa"] >= 3.0])
```

12. Quick Revision

`open()` opens files. `read()`, `readline()`, `readlines()` read data. `write()` writes data. `r`, `w`, `a` and `x` are core file modes. `csv.reader()` and `csv.DictReader()` read CSV. `csv.writer()` writes CSV. `json.dumps()/loads()` work with strings. `json.dump()/load()` work with files. CSV and JSON are important real-world data formats.

13. Day 7 Outcome

You can now work with text files, CSV files and JSON files. You can save structured records, read them back, filter data, calculate statistics, and build small data-processing programs.

Next target: Day 8